Algorithms
Assignment 4 Report

Warren Roberts
Student ID: 100450116
Student Email: wlr006@uark.edu
Late Days Used: 1 (3 total across assignments)
University of Arkansas

Purpose

The purpose of this assignment was to implement several important algorithms, including sorting algorithms, graph algorithms, and a balancing tree structure. This helped me better understand how graphs can be used for sorting and finding efficient paths. This assignment required implementing Heap Sort, Merge Sort, Topological Sort, Prim's Algorithm, Kruskal's Algorithm, Dijkstra's Algorithm, and the Red-Black Tree data structure.
These algorithms are important because they are widely used to process data efficiently, particularly for graph processing, search, and optimization. This assignment also helped reinforce how these algorithms work and why efficiency matters when dealing with large data/input.

Approach & Implementation

The base code already included the Graph and Disjoint Set classes, so my main focus was implementing each required algorithm correctly and making sure they worked with the given structure. I worked through each file step by step and tested often to make sure everything was working before moving on. The given test sets made it very easy to test and know what needed further implementation or reworking.

Heap Sort Heap Sort was implemented using a max-heap structure. The array is first converted into a max-heap, where the largest value is placed at the root. Then, the root is repeatedly swapped with the last element and removed from the heap while maintaining the heap property.

Merge Sort Merge Sort was implemented using a recursive divide-and-conquer approach. The array is split into smaller subarrays, sorted recursively, and then merged back together in sorted order.

Topological Sort Topological Sort was implemented using Kahn's Algorithm. This involves calculating the in-degree of each vertex and repeatedly removing nodes with zero in-degree while updating the graph.

Prim's Algorithm Prim's Algorithm was implemented using a min-heap to efficiently select the smallest edge. The algorithm starts from an initial node and grows the Minimum Spanning Tree by adding the smallest valid edge at each step.

Copyright © 2026, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

```python
from typing import List


# Function to perform heap sort
def heap_sort(arr: List[int]) -> None:
    # compute arr length once
    n = len(arr)

    # build the max heap
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)

    # heap sort
    for i in range(n - 1, 0, -1):
        # move max to end
        arr[0], arr[i] = arr[i], arr[0]

        # restore & maintain heap
        heapify(arr, i, 0)



# Helper function to maintain the max-heap property
def heapify(arr: List[int], n: int, i: int) -> None:
    # get the index and assume it is the largest,
    # its left child, and its right child
    largest, left, right = i, 2 * i + 1, 2 * i + 2

    # if left child is larger than index, update it
    if left < n and arr[left] > arr[largest]:
        largest = left

    # if right child is larger than index, update it
    if right < n and arr[right] > arr[largest]:
        largest = right

    # if largest is not the parent, swap and continue heapifying
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)

```

Figure 1: Heap Sort Implementation

Kruskal's Algorithm Kruskal's Algorithm was implemented using a Disjoint Set to prevent cycles. All edges are sorted using merge sort, and edges are added to the MST only if they connect different sets.

Dijkstra's Algorithm Dijkstra's Algorithm was implemented using a priority queue to always expand the shortest known path. Distances are updated whenever a shorter path is found, and the final path is reconstructed using a trace array.

```python
from typing import List, TypeVar

T = TypeVar("T")

def merge(array: List[T], l: int, m: int, r: int) -> None:
    # get the left and right side
    left = array[l:m + 1]
    right = array[m + 1:r + 1]

    # pointers for left, right, & merged array
    i, j, k = 0, 0, l

    # merge until one side runs out
    while i < len(left) and j < len(right):
        # compare elements from both halves
        if left[i] <= right[j]:
            # set the smaller element to the
            # correct position in given array
            array[k] = left[i]
            i += 1
        else:
            array[k] = right[j]
            j += 1
        k += 1

    # if left and right are not the same size,
    # one will finish first, so this will handle
    # the rest of left's elements.
    while i < len(left):
        array[k] = left[i]
        i += 1
        k += 1

    while j < len(right):
        array[k] = right[j]
        j += 1
        k += 1


def msort(array: List[T], l: int, r: int) -> None:
    if l < r:
        m = (l + r) // 2                # get middle index
        msort(array, l, m)
        msort(array, m + 1, r)          # recursively sort left & right side
        merge(array, l, m, r)           # combine those halves into one section
```

Figure 2: Merge Sort Implementation

Red-Black Tree The Red-Black Tree was implemented to maintain a balanced binary search tree. After inserting a node, the tree is adjusted using rotations and recoloring to maintain balance and ensure logarithmic height.

Time Complexity Each algorithm in this assignment has a known time complexity based on how it processes data.
Heap Sort runs in O(n log n) time because building the heap takes linear time and each removal step takes logarithmic time.
Merge Sort also runs in O(n log n) time since the array is divided into halves recursively and merged back together.
Topological Sort runs in O(V + E) time because each vertex and edge is processed once.
Prim's Algorithm runs in O(E log V) time when using a priority queue, since each edge may be pushed into the heap and extracted.
Kruskal's Algorithm runs in O(E log E) time due to sorting the edges using merge sort, along with near constant time operations from the Disjoint Set.
Dijkstra's Algorithm runs in O(E log V) time because it uses a priority queue to always process the next closest vertex.
The Red-Black Tree supports insertion and search operations in O(log n) time because the tree remains balanced after each insertion.

Challenges and Debugging

One of the main challenges was debugging Kruskal's Algorithm. Initially, the algorithm stopped too early due to an incorrect stopping condition, which caused the MST to be incomplete. Fixing this required ensuring the loop continued until n - 1 edges were added.

```python
from typing import List
from graph import Graph

def topological_sort(graph: Graph) -> List[int]:
    # Get the number of vertices in the graph
    n = graph.get_n()

    # Initialize in-degree array (number of incoming edges for each vertex)
    in_degree = [0] * n

    # for every vertex u
        # for each outgoing edge from u
    for u in range(n):
        for edge in graph.e[u]:
            in_degree[edge.v] += 1


    # go thru each node and add them
    # to the stack if in_degree = 0
    stack = []
    for i in range(n):
        if in_degree[i] == 0:
            stack.append(i)

    # final topological order (result)
    order = []

    # go thru all nodes w/ in_degree = 0
    while stack:
        u = stack.pop()
        order.append(u)

        # all nodes v that connect in direction towards u
        for edge in graph.e[u]:
            v = edge.v
            in_degree[v] -= 1

            # if v has no incoming edges
            if in_degree[v] == 0:
                stack.append(v)

    # if all nodes were not checked
    #  (meaning there is a cycle)
    if len(order) != n:
        return []
    else:
        return order
```

Figure 3: Topological Sort Implementation

Another issue was remembering to use merge sort for Kruskal instead of Python's built-in sorting. This required creating a custom merge sort specifically for sorting edges.
For Dijkstra's Algorithm, reconstructing the path correctly was difficult at first. I had to make sure the trace array was updated properly and that the destination was actually reached before returning the path.
The Red-Black Tree was the most difficult part of the assignment. Managing rotations and updating parent pointers correctly was tricky, and small mistakes caused the tree to become unbalanced. Once the rotation logic was corrected, the tree behaved as expected.

Results

All algorithms were successfully implemented and passed the provided unit tests.
The sorting algorithms correctly sorted all arrays. The graph algorithms produced correct results, with Prim's and Kruskal's algorithms matching in total MST cost. Dijkstra's Algorithm returned valid shortest paths, and the Red-Black Tree correctly handled insertion and searching.
The integrated demo also worked correctly, showing how multiple algorithms can work together in a single program.

Conclusion

This assignment helped reinforce several important algorithms and how they are used in practice. It also showed how

```python
from typing import List
import heapq

from graph import Edge, Graph

def construct_mst_prim(G: Graph) -> List[Edge]:
    edges = G.export_edges()  # Graph's edges (may include both directions)
    n = G.get_n()
    mst: List[Edge] = []
    heap: List[tuple[int, int, int, Edge]] = []
    # you can use heap to optimize the minimum searching

    # return if there are 0 vertices
    # create a set to track what nodes
    # have and have not been seen
    if n == 0: return mst
    visited = [False] * n

    # for step one we must
    # visit vertex 0 and set
    # push all edges into min-heap
    visited[0] = True
    for edge in G.e[0]:
        heapq.heappush(heap, (edge.w, edge.u, edge.v, edge))

    # add smallest valid edge repeating
    # until the mst has n - 1 edges.
    while heap and len(mst) < n - 1:
        w, u, v, edge = heapq.heappop(heap)

        if visited[v]:
            continue

        # if unvisited add this edge
        mst.append(edge)
        visited[v] = True

        # push all edges from this new vertex
        for next_edge in G.e[v]:
            if not visited[next_edge.v]:
                heapq.heappush(heap, (next_edge.w, next_edge.u, next_edge.v, next_edge))

    # after the loop, check for
    # a disconnected graph to ensure
    # that MST can include all vertices
    if len(mst) != n - 1:
        return []

    return mst
```

Figure 4: Prim's Algorithm Implementation

different algorithms can work together, especially in graph-based problems.
Overall, this assignment improved my understanding of efficiency, data structures, and debugging more complex algorithms.

```python
# helper function to merge two sorted halves of edges
def merge_edges(edges: List[Edge], l: int, m: int, r: int) -> None:
    # create left and right subarrays
    left = edges[l:m + 1]
    right = edges[m + 1:r + 1]

    i = 0
    j = 0
    k = l

    # merge based on edge weight
    while i < len(left) and j < len(right):
        if left[i].w <= right[j].w:
            edges[k] = left[i]
            i += 1
        else:
            edges[k] = right[j]
            j += 1
        k += 1

    # copy remaining elements from left if any
    while i < len(left):
        edges[k] = left[i]
        i += 1
        k += 1

    # copy remaining elements from right if any
    while j < len(right):
        edges[k] = right[j]
        j += 1
        k += 1


# helper function to perform merge sort on edges
def merge_sort_edges(edges: List[Edge], l: int, r: int) -> None:
    # recursively divide the list into halves
    if l < r:
        m = (l + r) // 2
        merge_sort_edges(edges, l, m)
        merge_sort_edges(edges, m + 1, r)
        merge_edges(edges, l, m, r)
```

Figure 5: Kruskal's Algorithm Implementation (part 1)

```python
def construct_mst_kruskal(G: Graph) -> List[Edge]:
    edges = G.export_edges()  # Graph's edges
    # DisjointSet djs(G.n);
    # Use Disjoint Set to check whether two vertices are on the same set
    # Usage: Check djs.is_on_same_set(u, v); Check is u and v is on the same set or not
    # djs.join(u, v); Join sets of u and v into the same set
    mst: List[Edge] = []
    djs = DisjointSet(G.get_n())

    n = G.get_n()

    # sort edges in weight ascending order using merge sort,
    # then we will handle each edge in that order
    if len(edges) > 1:
        merge_sort_edges(edges, 0, len(edges) - 1)

    for edge in edges:
        u, v = edge.u, edge.v

        # if u & v are not connected, add an edge
        if not djs.is_on_same_set(u, v):
            mst.append(edge)
            djs.join(u, v)

        # when MST has n - 1 edges
        if len(mst) == n - 1:
            break

    # after the loop, check for
    # a disconnected graph to ensure
    # that MST can include all vertices
    if len(mst) != n - 1:
        return []

    return mst
```

Figure 6: Kruskal's Algorithm Implementation (part 2)

```python
import heapq
from typing import List

from graph import Graph

def search_shortest_path(G: Graph, start: int, destination: int) -> List[int]:
    G.reset()
    n = G.get_n()
    G.dist[start] = 0
    # use heap to optimize the minimum searching
    heap: List[tuple] = [(0, start)]

    path: List[int] = []

    while heap:
        current_dist, u = heapq.heappop(heap)

        # skip if vertex has been visited
        if G.is_visited(u):
            continue

        G.set_visited(u)

        # stop at destomatopm
        if u == destination:
            break

        # check all of u's outgoing edges
        for edge in G.e[u]:
            v = edge.v
            w = edge.w

            # update distance if we find
            # a shorter path is found
            if not G.is_visited(v) and current_dist + w < G.dist[v]:
                G.dist[v] = current_dist + w
                G.set_trace(v, u)
                heapq.heappush(heap, (G.dist[v], v))

    # now if the destination is unreachable
    if G.dist[destination] == 10**9:
        return []

    # reconstruct shortest path by
    # following the traces backwards
    current = destination
    while current != -1:
        path.append(current)
        current = G.trace(current)

    path.reverse()
    return path
```

Figure 7: Dijkstra's Algorithm Implementation

```python
class RedBlackTree:
    def __init__(self):
        # tree starts empty
        self.root: Optional[RBNode] = None

    def rotate_left(self, root: Optional[RBNode], ptr: Optional[RBNode]) -> None:
        # performs a left rotation around ptr
        if ptr is None or ptr.right is None:
            return

        right_child = ptr.right
        ptr.right = right_child.left

        # if right_child had a left subtree, connect it to ptr
        if right_child.left is not None:
            right_child.left.parent = ptr

        # move right_child up into ptr's old position
        right_child.parent = ptr.parent
        if ptr.parent is None:
            self.root = right_child
        elif ptr == ptr.parent.left:
            ptr.parent.left = right_child
        else:
            ptr.parent.right = right_child

        # put ptr on the left of right_child
        right_child.left = ptr
        ptr.parent = right_child

    def rotate_right(self, root: Optional[RBNode], ptr: Optional[RBNode]) -> None:
        # performs a right rotation around ptr
        if ptr is None or ptr.left is None:
            return

        left_child = ptr.left
        ptr.left = left_child.right

        # if left_child had a right subtree, connect it to ptr
        if left_child.right is not None:
            left_child.right.parent = ptr

        # move left_child up into ptr's old position
        left_child.parent = ptr.parent
        if ptr.parent is None:
            self.root = left_child
        elif ptr == ptr.parent.left:
            ptr.parent.left = left_child
        else:
            ptr.parent.right = left_child

        # put ptr on the right of left_child
        left_child.right = ptr
        ptr.parent = left_child
```

Figure 8: Red-Black Tree Implementation (part 1)

```python
    def fix_violation(self, root: Optional[RBNode], ptr: Optional[RBNode]) -> None:
        # fixes red-black tree violations after insertion
        # the main violation handled here is:
        # a red node cannot have a red parent

        while (ptr is not None and ptr != self.root and ptr.parent is not None and ptr.parent.color == "RED"):
            parent = ptr.parent
            grandparent = parent.parent

            if grandparent is None:
                break

            # case: parent is left child of grandparent
            if parent == grandparent.left:
                uncle = grandparent.right

                # case 1: uncle is red
                # recolor parent and uncle black, grandparent red,
                # then continue fixing upward
                if uncle is not None and uncle.color == "RED":
                    grandparent.color = "RED"
                    parent.color = "BLACK"
                    uncle.color = "BLACK"
                    ptr = grandparent

                else:
                    # case 2: ptr is right child
                    # convert to case 3 by rotating left at parent
                    if ptr == parent.right:
                        self.rotate_left(self.root, parent)
                        ptr = parent
                        parent = ptr.parent

                    # case 3: ptr is left child
                    # rotate right at grandparent and recolor
                    self.rotate_right(self.root, grandparent)
                    parent.color = "BLACK"
                    grandparent.color = "RED"
                    ptr = parent

            # case: parent is right child of grandparent
            else:
                uncle = grandparent.left

                # case 1: uncle is red
                # recolor parent and uncle black, grandparent red,
                # then continue fixing upward
                if uncle is not None and uncle.color == "RED":
                    grandparent.color = "RED"
                    parent.color = "BLACK"
                    uncle.color = "BLACK"
                    ptr = grandparent

                else:
                    # case 2: ptr is left child
                    # convert to case 3 by rotating right at parent
                    if ptr == parent.left:
                        self.rotate_right(self.root, parent)
                        ptr = parent
                        parent = ptr.parent

                    # case 3: ptr is right child
                    # rotate left at grandparent and recolor
                    self.rotate_left(self.root, grandparent)
                    parent.color = "BLACK"
                    grandparent.color = "RED"
                    ptr = parent

        # the root of a red-black tree must always be black
        if self.root is not None:
            self.root.color = "BLACK"
```

Figure 9: Red-Black Tree Implementation (part 2)

```python
    def insert(self, data: int, x: int, y: int) -> None:
        # creates a new node to insert into the tree
        new_node = RBNode(data, x, y)

        # standard binary search tree insertion
        parent = None
        current = self.root

        while current is not None:
            parent = current
            if new_node.data < current.data:
                current = current.left
            elif new_node.data > current.data:
                current = current.right
            else:
                # do not insert duplicate values
                return

        # link the new node to its parent
        new_node.parent = parent

        if parent is None:
            # tree was empty, so new node becomes root
            self.root = new_node
        elif new_node.data < parent.data:
            parent.left = new_node
        else:
            parent.right = new_node

        # if inserted node is root, it must be black
        if new_node.parent is None:
            new_node.color = "BLACK"
            return

        # if there is no grandparent, there cannot be a violation yet
        if new_node.parent.parent is None:
            return

        # fix any red-black property violations caused by insertion
        self.fix_violation(self.root, new_node)

    def search(self, data: int) -> Optional[RBNode]:
        # searches for a node with the given data value
        temp = self.root

        while temp is not None:
            if data == temp.data:
                return temp
            if data < temp.data:
                temp = temp.left
            else:
                temp = temp.right

        # return none if value is not found
        return None
```

Figure 10: Red-Black Tree Implementation (part 3)

```
PS C:\Users\wpen\Desktop\school\20\algorithms\Assignment4\V&M-Sourcecode\V&M-Sourcecode\src> python main.py
 Perform unit test on heap sort
 Original array: 10 9 28 25 0 6 41 40 20 1 22 33 49 17 46 44 28 37 28 14
 Sorted array: 0 0 1 6 10 14 17 20 22 25 28 28 28 33 37 40 41 44 46 49
 Your sorting implementation is correct

 Perform unit test on Merge sort
 Original array: 35 8 35 4 3 13 5 1 44 4 27 22 42 7 15 10 39 20 20 24
 Sorted array: 1 3 4 4 5 7 8 10 15 20 22 24 26 27 32 35 35 39 42 44
 Your sorting implementation is correct


 Perform unit test on Topological sort
 Topological Sort: 5 2 3 4 1 0
 Topological Sort Implementation is correct.
 Perform unit test of the Prim's algorithm
 Total Cost of Minimum Spanning Tree: 16

 Total Cost of Minimum Spanning Tree: 9315

 Perform unit test of the Kruskal's algorithm
 Total Cost of Minimum Spanning Tree: 16

 Total Cost of Minimum Spanning Tree: 9315

 Perform unit test of the Dijkstra algorithm
 Path from 0 to 5: 0 -> 1 -> 2 -> 4 -> 5

 Path from SAEF to GRAD: SAEF -> BUCH -> LCAF -> GLAD2 -> PDTF -> SINF -> LPR111 -> WAHR -> SDMG2 -> AMB102 -> POST -> FPAC -> WATE -> AVSAGAS -> CORD1 -> PTSC -> MEMM2 -> ARMY -> GRAD

 Perform unit test of the Red-Black Tree

 ==================================================
 --- STARTING INTEGRATED ALGORITHM STACK DEMO ---
 ==================================================
 [Step 1] RB-Tree: Coordinates stored for Buildings 0, 1, and 2.
   -> Found Building 0 at (100, 100)
   -> Found Building 1 at (300, 300)
   -> Found Building 2 at (400, 150)

 [Step 2] Graph: Edges created with Euclidean weights:
   -> Edge (0,1) weight: 223
   -> Edge (1,2) weight: 250
   -> Edge (0,2) weight: 304
```

Figure 11: Final Program Output (first half)

```
[Step 3] Kruskal: Constructing Minimum Spanning Tree...
 -> Added Edge (0,1) w:223
 -> Added Edge (1,2) w:250
Total MST Cost: 473

[Step 4] Dijkstra: Calculating shortest path from 0 to 2...
 -> Shortest Path: 0 -> 2

[Step 5] HeapSort: Sorting traffic congestion scores...
 -> Original: 85 12 44 98 30
 -> Sorted:   12 30 44 85 98

=========================================================
--- INTEGRATED DEMO COMPLETE ---
=========================================================
```

Figure 12: Final Program Output (second half)